CA2 技术支持（Windows 版）：安装、复现与现场演示

> 给用 Windows 电脑的同学。按顺序做：第 1 节确认电脑能跑，第 2 节装好 VM 并登录，第 3 节装项目，第 4 节跑一遍完整实验，第 5–6 节练现场演示和 MiniEdit。出错先查第 8 节。现场演示的讲解和分工见 `docs/Live_Demo_Runbook_zh.pdf`。
>
> 约定：`$` 开头的是在 VM 的终端里输入；`mininet>` 开头的是在 Mininet 的 CLI 里输入。项目目录叫 `ceg5101-sdn-lab`。

1. 环境要求（Windows）

项目	要求	怎么检查
系统	Windows 10 或 11，64 位	设置 → 系统 → 关于
CPU	Intel 或 AMD，并且打开了硬件虚拟化（VT-x / AMD-V）	任务管理器 → 性能 → CPU，右下“虚拟化：已启用”。显示“已禁用”就要进 BIOS 打开（一般叫 Intel Virtualization Technology 或 SVM Mode）
内存	电脑 ≥ 8 GB，给 VM 4 GB	任务管理器 → 性能 → 内存
硬盘	空闲 ≥ 10 GB	此电脑
网络	安装那一步要能上网	装好以后跑实验不用网

要装的软件（都免费）：

软件	用途	下载
VirtualBox 7	运行官方 Mininet VM	https://www.virtualbox.org/wiki/Downloads，选 “Windows hosts”
MobaXterm Home	SSH 登录 VM，自带 X 窗口（MiniEdit 和总览图要用），左侧面板能拖拽传文件	https://mobaxterm.mobatek.net/download-home-edition.html
官方 Mininet VM 镜像	实验环境	https://github.com/mininet/mininet/releases/tag/2.3.0，下载 `mininet-2.3.0-210211-ubuntu-20.04.1-legacy-server-amd64-ovf.zip`（约 1 GB）

Windows 特有的坑：Hyper-V。 如果开着 Hyper-V、WSL2、“虚拟机平台”或“内核隔离 → 内存完整性”，VirtualBox 会变得很慢（窗口右下角出现一只绿色乌龟图标），甚至启动失败。演示用的那台电脑建议关掉：

1. 设置 → 隐私和安全性 → Windows 安全中心 → 设备安全性 → 内核隔离 → 关闭“内存完整性”。
2. 控制面板 → 程序 → 启用或关闭 Windows 功能 → 取消勾选 “Hyper-V”、“虚拟机平台”、“Windows 虚拟机监控程序平台”。
3. 以管理员身份打开 PowerShell，运行 `bcdedit /set hypervisorlaunchtype off`，然后重启。

（以后想恢复 WSL2，就把这些改回来，再运行 `bcdedit /set hypervisorlaunchtype auto`。）

我们已经实测通过的环境：官方 Mininet 2.3.0 VM（Ubuntu 20.04.1，内核 5.4，OVS 2.13 内核 datapath，Python 3.8），4 个 CPU、4 GB 内存。安装脚本一次跑通，18 个单元测试全过，完整实验和现场演示脚本都跑通。

2. 装 VM、登录、传文件（Windows）

2.1 导入 VirtualBox

1. 解压下载的 zip，得到 `.ovf` 和 `.vmdk`。
2. VirtualBox → 管理 → 导入虚拟电脑 → 选 `.ovf` → 导入。
3. 选中这台 VM → 设置：
    - 系统 → 主板：内存 4096 MB；处理器：2–4 个。
    - 网络 → 网卡 1：连接方式 NAT → 高级 → 端口转发 → 点右边的 + 加一条：名称 `ssh`，协议 TCP，主机 IP 留空，主机端口 2222，子系统端口 22。
4. 启动 VM，等出现登录提示（第一次开机 1–2 分钟）。这个窗口不用管，最小化即可。

2.2 用 MobaXterm 登录（带图形转发）

1. MobaXterm → Session → SSH：
    - Remote host：`127.0.0.1`
    - 勾选 Specify username：`mininet`
    - Port：`2222`
    - Advanced SSH settings 里确认勾着 X11-Forwarding（默认就是）。
2. 点 OK，输入密码 `mininet`。看到 `mininet@mininet-vm:~$` 就登录成功了。
3. 测试图形转发：输入 `xterm &`，应该弹出一个小黑窗口（关掉即可）。

以后每次在左侧 Sessions 里双击这个会话就能登录。需要两个终端时，再双击一次同一个会话。

2.3 传文件

- 最简单：登录后，MobaXterm 左边会出现 VM 的文件列表（SFTP 面板），把 `ceg5101-sdn-lab_part1_report_code_results.zip` 直接拖进去，就放到了 VM 的家目录 `/home/mininet/`。
- 或者用 PowerShell：在压缩包所在文件夹里按住 Shift 右键 → 在此处打开 PowerShell，运行 `scp -P 2222 ceg5101-sdn-lab_part1_report_code_results.zip mininet@127.0.0.1:`。
- 从 VM 拿回结果：在 MobaXterm 左边的文件列表里找到 `ceg5101-sdn-lab/results/…`，右键 Download。

2.4 演示时的终端设置

- MobaXterm → Settings → Configuration → Terminal → Default font：字号调到 16。
- 演示时把 MobaXterm 窗口最大化（或按 F11 全屏），保证四格画面至少有 160 列宽。
- 总览图弹出的窗口会出现在 Windows 桌面上，看完按 Esc 关闭。

3. 把项目装进 VM

3.1 拷项目

用第 2 节最后介绍的方法，把 `ceg5101-sdn-lab_part1_report_code_results.zip` 拷进 VM 的家目录（part1 就够跑实验；part2 是演示材料，留在自己电脑上用浏览器打开）。然后在 VM 里：

```
$ sudo apt install -y unzip        # 官方 VM 一般已有
$ unzip ceg5101-sdn-lab_part1_report_code_results.zip
$ cd ceg5101-sdn-lab
```

3.2 一键安装（约 5–10 分钟，需要联网）

```
$ bash scripts/setup_ubuntu.sh
```

它会装 OVS、hping3、iperf、tcpdump、ethtool、python3-tk，并在 `~/ryu-venv` 里装好 Ryu 4.34。最后打印两行 `export`，照着设置（建议写进 `~/.bashrc`，以后不用每次输入）：

```
$ echo 'export RYU_MANAGER=$HOME/ryu-venv/bin/ryu-manager
PLOT_PYTHON=$HOME/ryu-venv/bin/python' >> ~/.bashrc
$ source ~/.bashrc
```

3.3 自测（每一项都必须通过）

```
        $ $RYU_MANAGER --version                          # 应输出 ryu-manager
4.34
        $ ~/ryu-venv/bin/python -m unittest discover -s tests   # 应显示 Ran 18
tests ... OK
        $ python3 analysis/expected_paths.py              # h1->h6 应为 s1-s2-s6
(19)
        $ sudo mn --test pingall                          # Mininet 自带测试，0%
dropped
        $ sudo mn -c                                      # 清理
```

4. 完整复现实验（约 5 分钟）

```
        $ cd ~/ceg5101-sdn-lab
        $ sudo -E python3 experiments/run_experiment.py
```

- `-E` 不能少，它把 `RYU_MANAGER` 等环境变量带进 sudo。
- 脚本会自己启动控制器和 Mininet，依次跑 case a、b、c，最后画图。屏幕上会打印 `attack_start` / `attack_end` 等时间点。
- 有链路时延时，脚本会自动用 `CEG_WARMUP=12`（屏幕上会打印一行 `using CEG_WARMUP=12`）。这一步很重要，否则 h1 会被误封，原因见第 8 节。
- 结果在 `results/<日期时间>/`。最重要的是 `summary.md` 和 `figures/fig1–fig5`。

4.1 怎么判断结果对不对

打开 `results/<日期时间>/summary.md`，和我们的结果比（`results/vm-delay-abc/summary.md`）：

检查项	应该看到
blocked	三个 case 都是 yes（a、b 封 h2，c 封 h5）
anomalous links	a：s2-s3；b：s2-s6；c：s3-s5, s3-s6
h1→h6 path during	只有 b 变成 s1-s2-s3-s6，a 和 c 保持 s1-s2-s6
detect delay	大约 1.5–4 秒（每次会有差别，取决于洪泛落在采样周期的哪个位置）
Events 表	b 里有两次 h1→h6 reroute：一次 anomaly（离开），一次 cost_update（回来）
丢包	只在检测前那几秒有丢包；在 VM 上会比报告里的容器数字小，这是正常的

4.2 扩展实验（可选，每个约 2 分钟）

```
        $ sudo -E env CEG_BETA=2 python3 experiments/run_experiment.py --
scenarios b --baseline 30        # β 太小，不改路由
        $ sudo -E env CEG_COST_OVERRIDE=5-6:8 python3
experiments/run_experiment.py --scenarios c --baseline 30   # 让 case c 真的走
s5-s6
        $ sudo -E python3 experiments/run_experiment.py --scenarios "" --
baseline 30 --fail-link s2-s6 --fail-for 20 --recover 30   # 断链
```

4.3 只重新画图

```
        $ $PLOT_PYTHON analysis/plot_results.py results/<日期时间>
```

5. 现场演示

5.1 一条命令的完整演示（主方案）

```
        $ cd ~/ceg5101-sdn-lab
        $ bash demo/live_demo.sh          # case a、b、c，约 5–6 分钟
        $ bash demo/live_demo.sh quick    # 只跑 case b，约 2.5 分钟
```

终端分成四格：左上实验进度、左下控制器事件（BLOCK / REROUTE / UNBLOCK）、右上状态板（当前路径、链路代价、各源速率）、右下 h6 收到的每秒吞吐和丢包。跑完左上自动打印本次的结果表，再用它提示的 `python3 demo/show_figure.py results/live-…` 打开总览图。讲解台词、分工、时间安排都在 `docs/Live_Demo_Runbook_zh.pdf`。

- 终端要够大：全屏后至少 160 列、45 行（字号 16 左右）。
- 退出 tmux 画面：按 Ctrl-b，再按 d。中途出错：`sudo mn -c` 后重跑。

5.2 手动命令卡（老师要求一步步敲命令时用）

开两个终端，都先 `cd ~/ceg5101-sdn-lab`。演示前先 `sudo mn -c` 清理一次。

终端 1：控制器

```
        $ CEG_WARMUP=12 CEG_LOG_DIR=results/live/controller $RYU_MANAGER --
observe-links controller/integrated_controller.py 2>&1 | grep --line-buffered
EVENT
```

终端 2：Mininet

```
        $ sudo mn --custom topology/ceg_mesh_topo.py --topo meshTopo --link tc \
              --controller remote,ip=127.0.0.1,port=6633 --switch
ovsk,protocols=OpenFlow13 --mac
```

等终端 1 出现 `topology_ready`（约 10–25 秒），然后在 Mininet CLI 里按顺序输入（h1 的 iperf 要紧跟着 pingall，中间不要停太久）：

```
mininet> pingall
mininet> h6 iperf -s -u -i 1 > /tmp/h6.txt &
mininet> h1 iperf -u -c 10.0.0.6 -b 10M -t 300 > /dev/null &
mininet> h2 ping -i 1 10.0.0.4 > /dev/null &
mininet> sh sleep 30
mininet> h2 hping3 --icmp --flood -d 1000 10.0.0.6 > /dev/null 2>&1 &
```

几秒后终端 1 应出现 `[EVENT] block h2->h6` 和 `[EVENT] reroute h1->h6 ... 's1-s2-s3-s6'`。接着展示：

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s2 | grep 5000        # drop 规则，n_packets 在涨
mininet> h6 timeout 3 tcpdump -n -i h6-eth0 icmp and src host 10.0.0.2   # 0 packets captured
mininet> sh tail -5 /tmp/h6.txt                                       # h1->h6 每秒 0% 丢包
mininet> h2 pkill hping3                                              # 停止攻击，约 14 秒后出现 unblock 和回切
mininet> exit
```

结束后 `sudo mn -c`。

60 秒规则：洪泛开始后 60 秒内没看到 block，就别在现场调试了，直接切回幻灯片第 8 页的录屏。

6. MiniEdit（Task 1）

6.1 先修一个 Mininet 2.3.0 自带的 bug（只做一次）

不修的话，File → Save 会存出 0 字节的空文件。下面两行对官方 VM 和 apt 装的 Mininet 都适用：

```
        $ ME=$(python3 -c "import
mininet,os;print(os.path.dirname(mininet.__file__))")/examples/miniedit.py; echo
$ME
        $ sudo sed -i.orig "s/open(fileName, 'wb')/open(fileName, 'w')/" $ME
```

官方 VM 的家目录里还有一份 `~/mininet/examples/miniedit.py`，如果你用那一份启动，也要同样改一次。

6.2 启动

用第 2 节的方式登录，必须带图形转发（`-Y`）。

终端 1（控制器）：

```
        $ cd ~/ceg5101-sdn-lab
        $ CEG_LOG_DIR=results/miniedit/controller $RYU_MANAGER --observe-links
controller/integrated_controller.py
```

终端 2（MiniEdit）：

```
        $ ME=$(python3 -c "import
mininet,os;print(os.path.dirname(mininet.__file__))")/examples/miniedit.py
        $ sudo -E XAUTHORITY=$HOME/.Xauthority python3 $ME
```

6.3 搭拓扑（约 20 分钟，照着 **`presentation/video/miniedit_build.mp4`** 的顺序）

1. 工具栏选 Switch，放 s1–s6（上排 s1 s2 s3，下排 s4 s5 s6）；选 Host，放 h1–h6（h1–h3 在上，h4–h6 在下）。
2. 选 NetLink，从每台交换机拖到它的主机，再连 11 条交换机链路。
3. 选 Select，右键每条链路 → Properties：Bandwidth 填 `40`，Delay 填带单位的值：
   - 主机链路都是 `15ms`
   - s1-s2 `5ms`，s2-s3 `10ms`，s4-s5 `15ms`，s5-s6 `20ms`，s1-s4 `8ms`，s2-s5 `12ms`，s3-s6 `7ms`，s1-s5 `18ms`，s2-s4 `6ms`，s2-s6 `14ms`，s3-s5 `9ms`
4. 右键每台主机 → Properties：IP Address 填 `10.0.0.i/24`。
5. 选 Controller 放一个 c0，用 NetLink 从 c0 连到 6 台交换机（红色虚线）。右键 c0 → Properties：Controller Type 改成 Remote Controller（IP 127.0.0.1、端口 6633 是默认值）。
6. Edit → Preferences：去掉 OpenFlow 1.0，勾上 OpenFlow 1.3 和 Start CLI。
7. File → Save，存成 `~/ceg5101-sdn-lab/miniedit/my_topology_manual.mn`。
8. 截图：完整画布。

9. 点左下角 Run，等终端 1 出现 `topology_ready`，在终端 2 的 `mininet>` 里输入 `pingall`（应为 0% dropped，30/30），再输入 `h1 ping -c 4 h6`（约 100 ms）。截图。
10. 先在 CLI 里输入 `exit`，再点 Stop。

6.4 和我们的版本对照

```
        $ mnkey() { python3 -c "import json,sys;d=json.load(open(sys.argv[1]));
[print(*sorted([l['src'],l['dest']]),l['opts'].get('bw'),l['opts'].get('delay'))
for l in d['links']];[print(h['opts']['hostname'],h['opts'].get('ip')) for h in
d['hosts']]" "$1" | sort; }
        $ diff <(mnkey miniedit/my_topology.mn) <(mnkey
miniedit/my_topology_manual.mn) && echo 一致
```

没有输出差异、最后打印“一致”，就说明链路参数和主机 IP 完全相同。

7. 打开演示材料、改报告

- 幻灯片：用 Chrome 或 Edge 打开 `presentation/slides/CEG5101_CA2_Group5.html`（单文件，可以单独拷走）。按 → 或空格翻页，N 显示讲稿，P 打开演讲者视图，数字 + 回车跳页。
- 讲稿：`presentation/script/speaker_script.pdf`。题库：`docs/QA_Bank.pdf`。速读：`docs/Project_Quick_Guide_zh.pdf`。
- 改报告：编辑 `report/report.tex`，在装了 TeX 的电脑上执行 `cd report && latexmk -pdf report.tex`。不想装 TeX，可以把 `report/` 文件夹上传到 Overleaf 编译。
- 重新生成幻灯片用的图：`$PLOT_PYTHON presentation/scripts/plot_slides.py`。

8. 常见问题

现象	原因	怎么办
`setup_ubuntu.sh` 卡在 apt 或 pip	VM 上不了网	确认 VM 网卡是 NAT；在 VM 里 `ping 8.8.8.8` 测试；学校网络可能要换手机热点
`ryu-manager` 报 eventlet 或 `ALREADY_HANDLED` 错误	Python 版本太新，或 eventlet 版本不对	只用脚本装的 `~/ryu-venv/bin/ryu-manager`，不要用系统自带的
`run_experiment.py` 说找不到 ryu-manager	sudo 丢了环境变量	用 `sudo -E`，并确认 `echo $RYU_MANAGER` 有输出
`Address already in use` / 端口 6633 被占用	上一次的控制器还在	`sudo pkill -f ryu-manager`，再 `sudo mn -c`
`pingall` 不通	控制器还没发现全部 11 条链路，或没开 OpenFlow 1.3	等终端出现 `topology_ready` 再 ping；确认控制器带了 `--observe-links`，交换机是 `protocols=OpenFlow13`
h1 一开始就被封，h1→h6 吞吐一直为 0（events.csv 里有 `block,h1`）	warm-up 太短：带时延时 pingall 变慢，h1 的 iperf 在 warm-up 结束后才开始	用 `run_experiment.py`（会自动设 12）；手动演示时控制器前面加 `CEG_WARMUP=12`，并且 iperf 紧跟 pingall 启动
某个 case 没被检测到	攻击源没有基线，或者还在 warm-up	确认有背景 ping，基线阶段至少 30 秒；查 `controller/sources.csv`
`Specified qdisc kind is unknown`	系统没有 netem（容器里常见）	官方 VM 不会出现；容器里脚本会自动去掉时延
MiniEdit 或总览图报 `cannot connect to X server` / `couldn't connect to display`	没有 X 转发	按第 2 节用带 `-Y` 的方式登录；sudo 时带上 `XAUTHORITY=$HOME/.Xauthority`
MiniEdit 保存后文件是 0 字节，终端显示 `a bytes-like object is required`	Mininet 2.3.0 的 bug	做第 6.1 节的修改，重启 MiniEdit 再保存
MiniEdit 打开 `.mn` 时报 `dict_items + dict_items`	这个版本读 preferences 的 bug	用 `miniedit/ceg_mesh_topo.mn`；自己存的文件删掉 `"application"` 那一段

现象	原因	怎么办
实验中途出错，之后什么都不通	残留的 Mininet 进程和网卡	`sudo mn -c`，`sudo pkill -f ryu-manager`，再重跑
幻灯片里视频不播放	浏览器不支持 H.264，或在 App 内预览	用 Chrome / Edge 直接打开 HTML 文件
压缩包打不开	文件太大或下载不完整	用电脑下载后再解压；也可以分别下载 part1 和 part2
`tmux: command not found`	旧版安装脚本没装 tmux	`sudo apt install -y tmux`

9. 实在不行时

- 实验数据我们已经有完整的一套：报告和幻灯片的所有数字都来自 `results/` 里的文件，不需要你们重跑。
- 自己跑一遍的意义是熟悉过程，被问到时能讲清楚、能现场演示。最低要求：第 3.3 节的自测全部通过，第 5 节的演示命令能走一遍。